

Resource Requests, Limits e Quota

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Distinguere tra **requests** e **limits** di CPU e memoria e il loro impatto sullo scheduling e sull'esecuzione
- Descrivere le tre classi QoS (**Guaranteed**, **Burstable**, **BestEffort**) e la loro priorità di eviction
- Configurare **LimitRange** per impostare valori default e massimi per container/Pod in un namespace
- Creare **ResourceQuota** per limitare il consumo totale di risorse in un namespace
- Spiegare la relazione tra requests/limits e il Node Autoscaler su ARO
- Diagnosticare Pod in stato **OOMKilled** o con CPU throttling

Teoria e Concetti Chiave

Requests vs Limits

Caratteristica	requests	limits
Significato	Risorse garantite dallo scheduler	Massimo consentito dal cgroup
Impatto scheduling	Lo scheduler usa requests per trovare un nodo con capacità sufficiente	Non influenza il placement
Comportamento CPU	Nodo garantisce questa quantità	CPU throttling se superato
Comportamento memoria	Nodo garantisce questa quantità	Container killed se superato (OOMKilled)
Obbligatorio	No (ma consigliato)	No (ma consigliato)

Unità di misura:

- **CPU:** **m** = millicores (1000m = 1 core). Es: **500m** = 0.5 core
- **Memoria:** **Mi** = mebibyte, **Gi** = gibibyte. Es: **256Mi**, **2Gi**

```
resources:
  requests:
    memory: "256Mi"    # ← scheduler riserva 256Mi sul nodo
    cpu: "250m"        # ← scheduler riserva 0.25 core sul nodo
  limits:
    memory: "512Mi"    # ← OOMKill se il container usa >512Mi
    cpu: "1000m"       # ← CPU throttling se usa >1 core
```

Classi QoS (Quality of Service)

Kubernetes assegna automaticamente una classe QoS a ogni Pod in base a requests e limits. Questa classe determina la **priorità di eviction** quando il nodo è sotto pressione di memoria:

Classe QoS	Condizione	Eviction priority	Uso consigliato
Guaranteed	requests = limits per tutti i container	Ultima ad essere evicted	Workload critici, database
Burstable	requests < limits (almeno 1 container con requests)	Media	Workload normali
BestEffort	Nessuna requests né limits configurata	Prima ad essere evicted (!)	Batch job non critici

⚠ **Attenzione:** Pod **BestEffort** sono i primi ad essere terminati sotto pressione. In produzione, tutti i Pod devono avere almeno le **requests** configurate.

LimitRange

LimitRange è un oggetto namespace-scoped che impone vincoli su requests e limits per singolo container o Pod:

Funzionalità:

- **Default values:** se un container non specifica requests/limits, vengono applicati i default del LimitRange
- **Valori massimi (max):** il container non può superare questi valori
- **Valori minimi (min):** il container deve dichiarare almeno questi valori
- **Ratio max/min:** limita il rapporto tra limit e request

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: myproject
spec:
  limits:
  - type: Container
    default:           # ← se non specificato nel container
      cpu: "500m"
      memory: "256Mi"
    defaultRequest:    # ← se non specificata la request
      cpu: "100m"
      memory: "128Mi"
    max:               # ← massimo consentito per container
      cpu: "2000m"
      memory: "2Gi"
    min:               # ← minimo obbligatorio
      cpu: "50m"
      memory: "64Mi"
```

ResourceQuota

ResourceQuota limita il **totale** di risorse consumabili in un namespace:

Risorsa	Descrizione
<code>requests.cpu</code>	Somma dei CPU requests di tutti i Pod
<code>requests.memory</code>	Somma dei memory requests
<code>limits.cpu</code>	Somma dei CPU limits
<code>limits.memory</code>	Somma dei memory limits
<code>count/pods</code>	Numero massimo di Pod
<code>count/services</code>	Numero massimo di Service
<code>persistentvolumeclaims</code>	Numero massimo di PVC
<code>requests.storage</code>	Storage totale richiesto da PVC

 **Suggerimento:** Se un namespace ha una **ResourceQuota** con `requests.cpu`, ogni Pod nel namespace **deve** dichiarare `resources.requests.cpu`. Altrimenti la creazione del Pod fallisce con errore `forbidden: failed quota`.

Relazione con Node Autoscaler

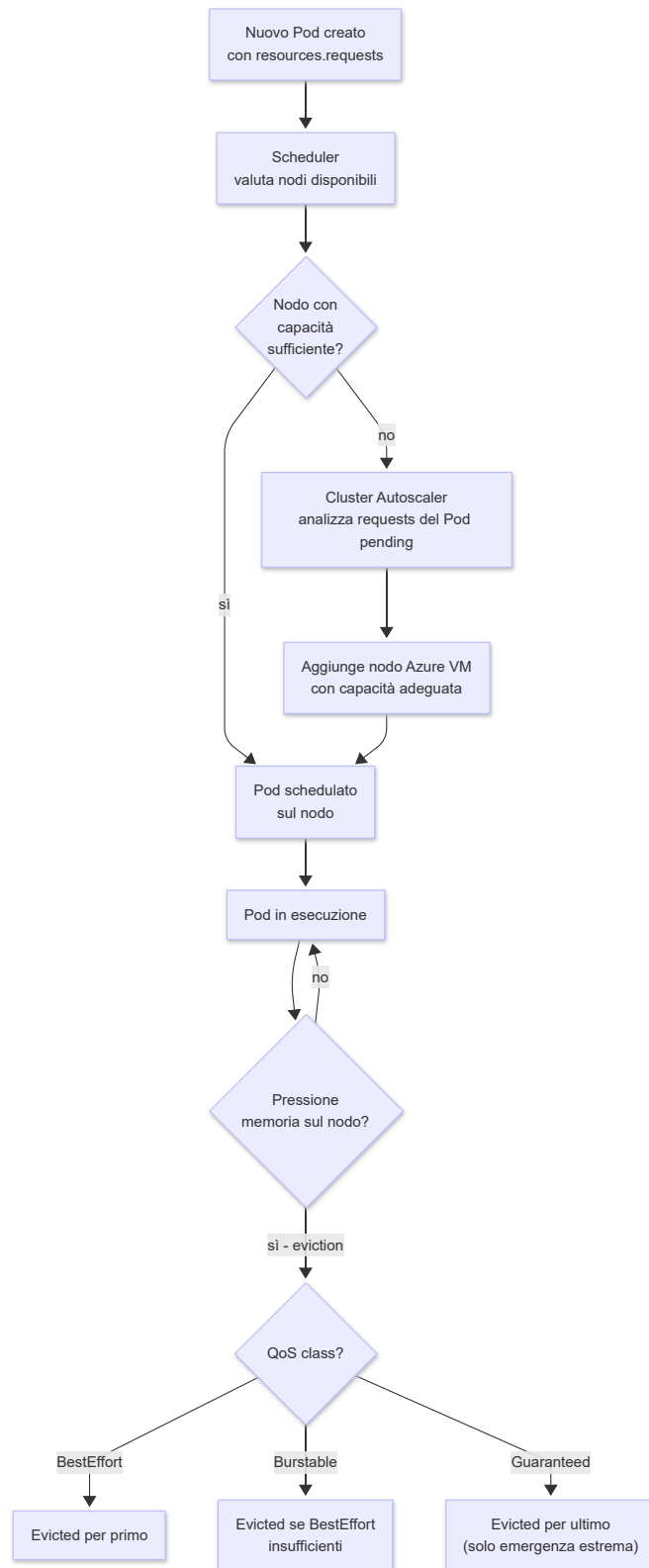
Su ARO, il **Cluster Autoscaler** scala i nodi in base alle requests (non ai limits):

```
Nuovo Pod creato con requests.memory: 4Gi
→ Scheduler non trova nodo con 4Gi liberi
→ Cluster Autoscaler aggiunge un nuovo nodo Azure VM
→ Pod schedulato sul nuovo nodo
```

Se i Pod non dichiarano requests, il Cluster Autoscaler non può calcolare correttamente il fabbisogno e potrebbe non scalare anche se i nodi sono effettivamente saturi.

Diagrammi

Flow di Scheduling con QoS Classes e Node Autoscaler



Comandi Pratici con Output Atteso

```
# Verificare la QoS class di un Pod
oc get pod myapp-6b4c9d-xz9kp -n myproject \
  -o jsonpath='{.status.qosClass}'

# Output atteso:
# Burstable
```

```
# Creare un LimitRange con default per il namespace
oc apply -f - <<EOF
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: myproject
spec:
  limits:
  - type: Container
    default:
      cpu: "500m"
      memory: "256Mi"
    defaultRequest:
      cpu: "100m"
      memory: "128Mi"
    max:
      cpu: "4000m"
      memory: "4Gi"
EOF

# Output atteso:
# limitrange/default-limits created
```

```
# Verificare il LimitRange attivo
oc describe limitrange default-limits -n myproject

# Output atteso:
# Name:          default-limits
# Namespace:     myproject
# Type           Resource  Min    Max    Default Request  Default Limit  ...
# ----          -
# Container      cpu        -      4      100m             500m
# Container      memory     -      4Gi    128Mi            256Mi
```

```
# Creare una ResourceQuota per il namespace
oc apply -f - <<EOF
apiVersion: v1
kind: ResourceQuota
metadata:
  name: myproject-quota
  namespace: myproject
spec:
  hard:
    requests.cpu: "8"
    requests.memory: "16Gi"
    limits.cpu: "32"
    limits.memory: "64Gi"
    count/pods: "50"
    persistentvolumeclaims: "20"
    requests.storage: "200Gi"
EOF

# Output atteso:
# resourcequota/myproject-quota created
```

```
# Verificare l'utilizzo attuale vs quota
oc describe resourcequota myproject-quota -n myproject

# Output atteso:
# Name:                myproject-quota
# Namespace:           myproject
# Resource              Used      Hard
# -----
# count/pods            8        50
# limits.cpu            3500m    32
# limits.memory         7Gi      64Gi
# persistentvolumeclaims 3        20
# requests.cpu          1200m    8
# requests.memory       2Gi      16Gi
# requests.storage      50Gi     200Gi
```

```
# Verificare se un Pod è stato OOMKilled (memoria limit superata)
oc get pods -n myproject -o json | \
  jq '.items[] |
  select(.status.containerStatuses[].lastState.terminated.reason=="OOMKilled") |
  .metadata.name'

# Output atteso (se ci sono Pod OOMKilled):
# "myapp-7f9d8b-abc12"
```

```
# Descrivere un Pod OOMKilled per vedere il motivo
oc describe pod myapp-7f9d8b-abc12 -n myproject | grep -A10 "Last State:"
```

```
# Output atteso:
# Last State:      Terminated
# Reason:         OOMKilled
# Exit Code:      137
# Started:        Sat, 24 May 2026 10:00:00 +0000
# Finished:       Sat, 24 May 2026 10:05:23 +0000
```

```
# Verificare se CPU throttling è attivo (metriche cgroup)
oc exec deployment/myapp -n myproject -- \
  cat /sys/fs/cgroup/cpu/cpu.stat | grep throttled
```

```
# Output atteso:
# nr_throttled 1423          ← numero di periodi con throttling
# throttled_time 56789012345 ← nanosecondi totali di throttling
```

Esempi YAML Completi

LimitRange + ResourceQuota + Deployment Conforme

```
# LimitRange: imposta default e vincoli per tutti i container del namespace
apiVersion: v1
kind: LimitRange
metadata:
  name: team-default-limits
  namespace: team-project
  labels:
    managed-by: platform-team
spec:
  limits:
  - type: Container
    # Default applicati se il container non specifica requests/limits
    defaultRequest:
      cpu: "100m"           # ← default request CPU
      memory: "128Mi"       # ← default request memoria
    default:
      cpu: "500m"           # ← default limit CPU
      memory: "256Mi"       # ← default limit memoria
    # Vincoli massimi: la creazione del container fallisce se li supera
    max:
      cpu: "4000m"          # ← massimo 4 core per container
      memory: "4Gi"         # ← massimo 4 GiB per container
    # Vincoli minimi
    min:
      cpu: "50m"            # ← minimo 50m CPU per container
      memory: "64Mi"        # ← minimo 64 MiB per container
  - type: Pod
    max:
      cpu: "8000m"          # ← somma CPU di tutti i container in un
Pod: max 8 core
      memory: "8Gi"         # ← somma memoria: max 8 GiB per Pod
---
# ResourceQuota: limita il consumo totale nel namespace
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-project-quota
  namespace: team-project
  labels:
    managed-by: platform-team
spec:
  hard:
    # Limiti su risorse di calcolo
    requests.cpu: "16"      # ← somma requests CPU di tutti i Pod: max
16 core
    requests.memory: "32Gi" # ← somma requests memoria: max 32 GiB
    limits.cpu: "64"        # ← somma limits CPU: max 64 core
```

```

limits.memory: "128Gi"           # ← somma limits memoria: max 128 GiB
# Limiti su oggetti Kubernetes
count/pods: "100"                # ← max 100 Pod nel namespace
count/services: "20"            # ← max 20 Service
count/deployments.apps: "30"    # ← max 30 Deployment
count/configmaps: "50"          # ← max 50 ConfigMap
# Limiti su storage
persistentvolumeclaims: "30"    # ← max 30 PVC
requests.storage: "500Gi"       # ← storage totale richiesto: max 500 GiB
---
# Deployment conforme con requests e limits esplicitamente configurati
# QoS class: Burstable (requests < limits)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  namespace: team-project
  labels:
    app: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: app
          image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
          command: ["/bin/sh", "-c", "sleep 3600"]
          resources:
            requests:
              memory: "256Mi"      # ← scheduler riserva 256Mi × 3 repliche =
768Mi totali
              cpu: "200m"         # ← scheduler riserva 200m × 3 = 600m core
totali
            limits:
              memory: "512Mi"     # ← OOMKill se supera 512Mi
              cpu: "1000m"        # ← CPU throttling se supera 1 core
          # Con queste impostazioni:
          # QoS class = Burstable (requests < limits)
          # 3 repliche × requests.cpu=200m = 600m contribuisce alla ResourceQuota
          # 3 repliche × requests.memory=256Mi = 768Mi contribuisce alla
ResourceQuota

```

Pod Guaranteed (richiede requests = limits)

```
# Pod con QoS class Guaranteed: requests = limits per tutti i container
# Usare per workload critici che non devono essere evicted sotto pressione memoria
apiVersion: v1
kind: Pod
metadata:
  name: critical-db-pod
  namespace: team-project
  labels:
    app: critical-db
    qos-class: guaranteed          # ← label informativa
spec:
  containers:
  - name: postgres
    image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
    command: ["/bin/sh", "-c", "sleep 3600"]
    resources:
      requests:
        memory: "2Gi"             # ← requests = limits → Guaranteed
        cpu: "1000m"
      limits:
        memory: "2Gi"             # ← stesso valore di requests
        cpu: "1000m"              # ← stesso valore di requests
  # QoS class = Guaranteed
  # Kubernetes evicterà questo Pod solo come ultima risorsa
```

Note Specifiche per Azure ARO

ResourceQuota per Progetto ARO

Su ARO, il concetto di **Project** (namespace OpenShift) è il confine naturale per applicare ResourceQuota. Ogni team/applicazione dovrebbe avere il proprio Project con quota dedicata:

```
# Creare un nuovo progetto ARO con quota predefinita
oc new-project team-frontend

# Applicare la quota al progetto
oc apply -f team-quota.yaml -n team-frontend

# Verificare utilizzo quota in tutti i namespace con quota configurata
oc get resourcequota --all-namespaces
```

Requests/Limits e Node Autoscaler ARO

Su ARO, il Cluster Autoscaler è integrato con Azure VM Scale Sets. Le **requests** sono il segnale primario per il scale-out:

```
# Verificare la configurazione del Cluster Autoscaler su ARO
oc get clusterautoscaler cluster -o yaml | grep -E "maxNodes|minNodes|scaleDown"

# Verificare i MachineAutoscaler per ogni MachineSet
oc get machineautoscaler -n openshift-machine-api

# Output atteso:
```

# NAME	REF KIND	REF NAME	MIN	MAX	AGE
# aro-worker-eastus-1	MachineSet	aro-worker-eastus-1	1	5	10d
# aro-worker-eastus-2	MachineSet	aro-worker-eastus-2	1	5	10d

 **Suggerimento:** Configurare le **requests** in modo accurato è fondamentale su ARO: il Cluster Autoscaler calcola i nodi necessari in base alle **requests** dei Pod pending, non ai **limits**. Requests troppo basse → autoscaler non aggiunge nodi anche se i limiti sono superati.

Azure Monitor per Metriche CPU/Memoria su ARO

Su ARO, le metriche di utilizzo CPU e memoria dei container sono disponibili in **Azure Monitor** / Container Insights:

```
# Verificare l'utilizzo di CPU e memoria a livello Pod con oc top
oc top pods -n myproject --sort-by=memory

# Output atteso:
# NAME                                CPU(cores)    MEMORY(bytes)
# myapp-7f9d8b-abc12                  523m          498Mi        ← vicino al limit 512Mi →
# myapp-7f9d8b-def34                  187m          221Mi
# myapp-7f9d8b-ghi56                  201m          235Mi

# Verificare utilizzo a livello nodo
oc top nodes

# Output atteso:
# NAME                                CPU(cores)    CPU%    MEMORY(bytes)    MEMORY%
# aro-worker-eastus-1                2341m         58%     11Gi             71%
# aro-worker-eastus-2                1876m         47%     8Gi              52%
```

Diagnosi OOMKilled su ARO

```
# Trovare Pod con OOMKilled nell'ultima ora
oc get events -n myproject \
  --field-selector reason=OOMKilling \
  --sort-by='.lastTimestamp'

# Aumentare il memory limit di un Deployment dopo OOMKilled
oc set resources deployment/myapp \
  --limits=memory=1Gi \
  --requests=memory=512Mi \
  -n myproject

# Output atteso:
# deployment.apps/myapp resource requirements updated
```

Riepilogo dei Punti Chiave

- Le **requests** sono risorse **garantite** usate dallo scheduler per il placement; le **limits** sono il **massimo consentito** dal cgroup (CPU throttling o OOMKill se superate)
 - Le classi QoS determinano la priorità di eviction: **BestEffort** (senza requests/limits) è la prima terminata; **Guaranteed** (requests=limits) è l'ultima
 - **LimitRange** imposta automaticamente default di requests/limits per i container che non li dichiarano, e blocca la creazione di container che superano i valori massimi
 - **ResourceQuota** limita il consumo totale di risorse nel namespace; se attiva, ogni Pod **deve** dichiarare requests/limits (altrimenti la creazione fallisce)
 - Su ARO, il Cluster Autoscaler scala i nodi in base alle **requests** dei Pod pending: requests accurate = autoscaling efficiente
 - OOMKilled (exit code 137) indica che il memory limit è troppo basso; aumentare **limits.memory** e verificare con **oc top pods**
-

Domande di Verifica

1. Un container ha `requests.memory: 256Mi` e `limits.memory: 512Mi`. Quale QoS class viene assegnata al Pod?

- A) `Guaranteed`
- B) `BestEffort`
- C) `Burstable` ✓
- D) `Critical`

2. Il namespace `myproject` ha una `ResourceQuota` con `requests.cpu: "4"`. Un nuovo Pod viene creato senza `resources.requests.cpu`. Cosa accade?

- A) Il Pod viene creato con CPU request = 0
- B) Il Pod eredita automaticamente la request dal LimitRange
- C) La creazione del Pod fallisce con errore `forbidden: failed quota` ✓
- D) Il Pod viene creato in stato Pending finché non si libera CPU

3. Quale comando mostra l'utilizzo attuale di risorse rispetto alla quota configurata in un namespace?

- A) `oc get resourcequota`
- B) `oc describe resourcequota <name>` ✓
- C) `oc top namespace`
- D) `oc get limitrange`

4. Un Pod viene terminato con exit code 137. Quale è la causa più probabile?

- A) La `livenessProbe` ha fallito per `failureThreshold` volte
- B) Il container ha superato il `limits.memory` ed è stato OOMKilled ✓
- C) Il container ha superato il `limits.cpu` ed è stato terminato
- D) Lo scheduler ha rimosso il Pod per liberare risorse (eviction)

5. Su ARO, il Cluster Autoscaler aggiunge un nuovo nodo. Quale valore usa per determinare se è necessario un nuovo nodo?

- A) Il valore di `limits.cpu` e `limits.memory` dei Pod pending
- B) Il valore di `requests.cpu` e `requests.memory` dei Pod pending ✓
- C) L'utilizzo effettivo di CPU e memoria dei Pod in esecuzione
- D) Il numero totale di Pod nel cluster